



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Risolvere i problemi con la ricerca

Intelligenza Artificiale

Ingegneria Informatica

Docente

Domenico Fabio Savo

Riepilogo Agenti Intelligenti (Intro)

- Un **agente** è qualcosa che percepisce e agisce all'interno di un ambiente. La sua **funzione agente** specifica l'azione intrapresa in risposta a qualsiasi sequenza di percezioni.
- La **misura di prestazione** valuta il comportamento dell'agente in un ambiente operativo. Un **agente razionale** agisce in modo da massimizzare il valore atteso della misura di prestazione, data la sequenza percettiva fino a quel momento.
- Gli ambienti operativi possono essere classificati in base a molte proprietà significative. Possono essere completamente o parzialmente osservabili, deterministici o non deterministici, episodici o sequenziali, statici o dinamici, discreti o continui. Nella progettazione di un agente il primo passo deve sempre consistere nella specifica più dettagliata possibile dell'ambiente operativo.
- Il **programma agente** implementa la funzione agente. Esistono diversi schemi base per i programmi agente, che riflettono il tipo di informazione rappresentata esplicitamente e utilizzata nel processo decisionale. Lo schema più appropriato per un dato programma agente dipende dalla natura dell'ambiente.
- Gli **agenti reattivi semplici** rispondono direttamente alle percezioni, mentre gli **agenti reattivi basati su modello** mantengono uno stato interno per tener traccia degli aspetti del mondo che non sono visibili nelle percezioni correnti. Gli **agenti basati su obiettivi** agiscono per raggiungere tali obiettivi, e gli **agenti basati sull'utilità** cercano di massimizzare la "**contentezza**" attesa.



Agenti Risolutori di Problemi e Ricerca

Quando l'azione giusta da compiere non è subito evidente, un agente può avere la necessità di **guardare in avanti**, cioè considerare una sequenza di azioni che formano un **cammino** che porterà a uno stato obiettivo.

Un agente che adotta tale strategia è chiamato **agente risolutore di problemi** e il processo computazionale che effettua è una **ricerca**.



Soluzione del problema come ricerca in uno spazio degli stati

Molti problemi di IA hanno la **ricerca** (o controllo) come componente fondamentale.

Tali problemi si possono modellare come Problemi di Ricerca in uno **spazio degli stati** (Strategie di Ricerca).

Spazio degli stati: l'insieme di tutti gli stati raggiungibili dallo stato iniziale con una qualunque sequenza di operatori.

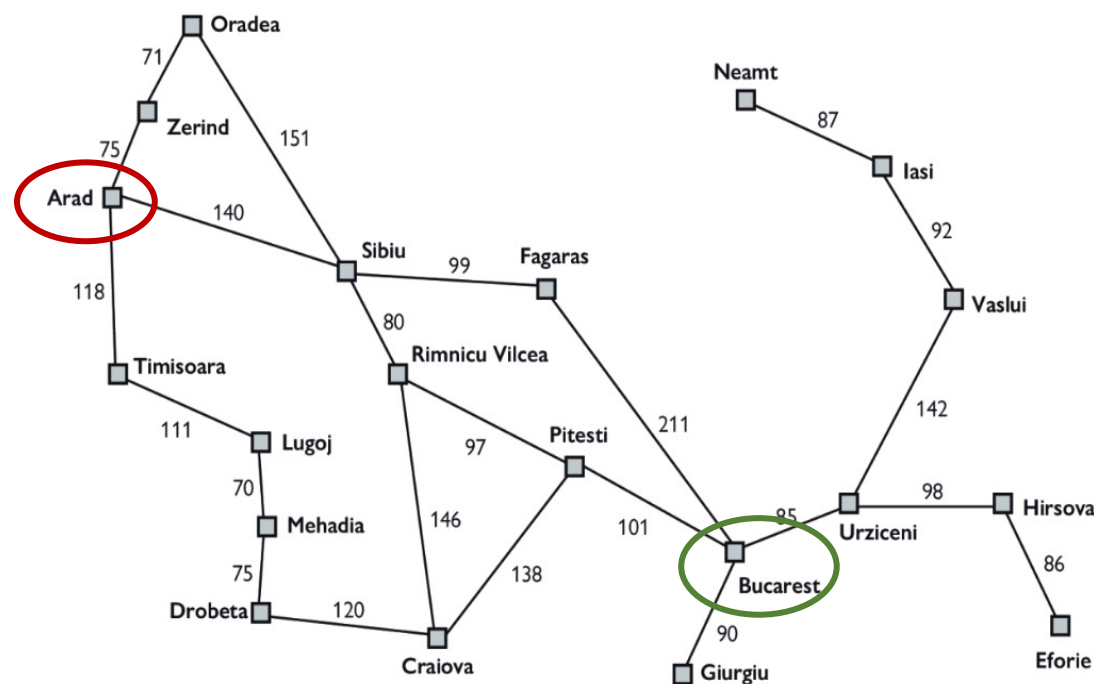
Lo Spazio degli Stati è Caratterizzato da:

- Uno **stato iniziale** in cui l'agente sa di trovarsi (non noto a priori);
- Un **insieme di azioni possibili** che sono disponibili da parte dell'agente (Operatori che trasformano uno stato in un altro o più formalmente una funzione successore $S(X)$ che riceve in ingresso uno stato e restituisce l'insieme degli stati raggiungibili).
- Un **cammino** è una sequenza di azioni che conduce da uno stato a un altro.



Esempio: un viaggio in Romania

Un agente che si trova nella città di **Arad** in Romania deve raggiungere **Bucarest**. La segnaletica stradale indica che ci sono tre strade che partono da Arad: una verso Sibiu, una verso Timisoara e una verso Zerind. Nessuna di queste è l'obiettivo, perciò l'agente, deve usare le conoscenze in suo possesso. In questo caso una **mappa**.



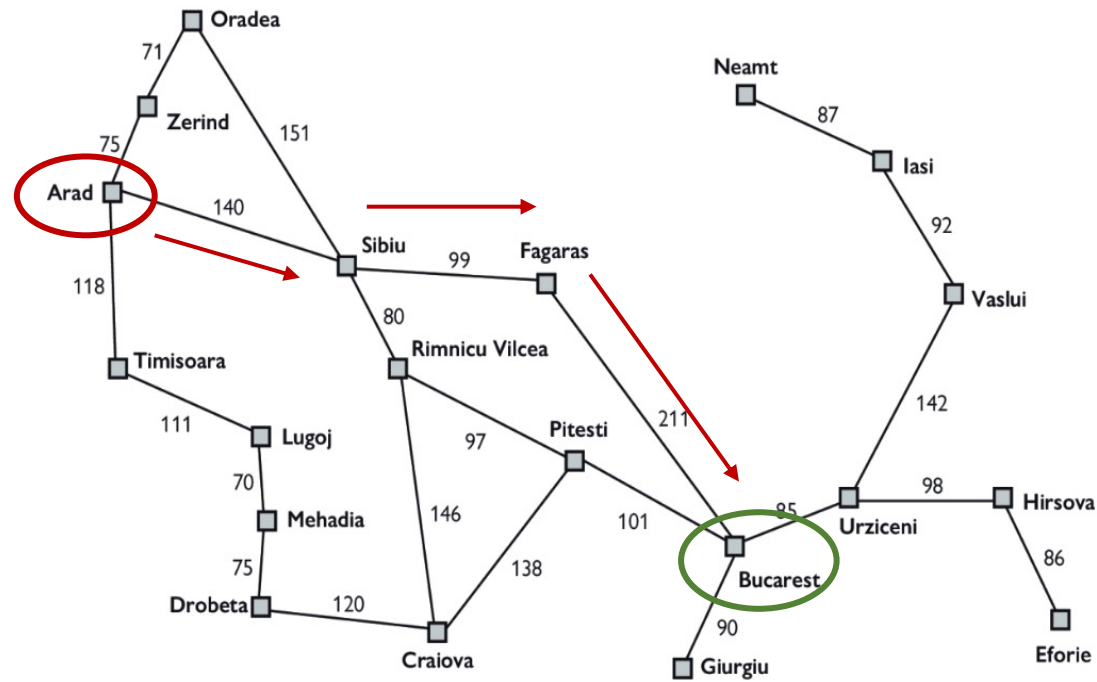
Processo di risoluzione del problema

- **Formulazione dell'obiettivo:** Gli obiettivi aiutano a organizzare il comportamento le azioni da considerare. Il nostro agente fissa come **obiettivo** di raggiungere Bucarest
- **Formulazione del problema:** l'agente elabora una descrizione degli stati e delle azioni necessarie per raggiungere l'obiettivo, ovvero un modello astratto della parte del mondo interessata. Per il nostro agente, un buon modello consiste nel considerare le azioni di viaggiare da una città a un'altra città adiacente, perciò l'unico fatto relativo allo stato del mondo che cambierà a causa di un'azione è la città corrente.
- **Ricerca:** prima di effettuare qualsiasi azione nel mondo reale, l'agente simula nel suo modello sequenze di azioni, continuando a cercare finché trova una sequenza che raggiunge l'obiettivo: tale sequenza si chiama **soluzione**. L'agente potrebbe simulare più sequenze che non raggiungono l'obiettivo, ma alla fine troverà una soluzione oppure troverà che non è possibile alcuna soluzione.
(Es: andare da Arad a Sibiu, poi a Fagaras ed infine a Bucarest)
- **Esecuzione:** l'agente ora può eseguire le azioni specificate nella soluzione, una per volta.



Esempio: un viaggio in Romania

Soluzione calcolata dal nostro agente



Ambiente osservabile, deterministico e noto

In un ambiente completamente **osservabile**, **deterministico** e **noto**, la soluzione di qualsiasi problema è **una sequenza fissata di azioni**.

Es: guidare fino a Sibiu, poi fino a Fagaras, poi fino a Bucarest.

Se il modello è corretto, una volta trovata una soluzione, l'agente può ignorare le sue **percezioni** durante la fase di Esecuzione dato che ha la garanzia che la soluzione lo condurrà all'obiettivo.

Nella teoria del controllo si parla in questo caso di **sistema ad anello aperto**, perché ignorando le percezioni si rompe il ciclo tra agente e ambiente. Se vi è la possibilità che il modello sia errato, o che l'ambiente non sia deterministico, l'agente sarebbe più sicuro usando un approccio ad **anello chiuso** in cui continua a monitorare le **percezioni**.



Problema di ricerca

Un **problema di ricerca** può essere definito formalmente come segue:

- Lo **Spazio degli Stati**: un insieme di possibili stati in cui può trovarsi l'ambiente.
- Lo **Stato Iniziale** in cui si trova l'agente inizialmente.
- Lo **Stato Obiettivo**: a volte lo stato obiettivo è uno solo, a volte è un insieme di stati, a volte l'obiettivo è **definito da una proprietà** che è soddisfatta da diversi stati.
- Le **azioni possibili** dell'agente definite dalla funzione **Azioni($\bullet$)**. Dato uno stato **s**, **Azioni(s)** restituisce un insieme finito di azioni che possono essere eseguite in **s**. Diciamo che ognuna di queste azioni è **applicabile in s**.
- Un **modello di transizione Risultato($\bullet, \bullet$)** che descrive ciò che fa ogni azione. **Risultato(s, a)** restituisce lo **stato** risultante dall'esecuzione dell'azione **a** nello stato **s**.
- Una funzione **Costo-Azione($\bullet, \bullet, \bullet$)** di **costo dell'azione**. **Costo-Azione(s, a, s')** restituisce il costo numerico di applicare l'azione **a** nello stato **s** per raggiungere lo stato **s'**. Una funzione di costo riflette la **misura di prestazione**.



Soluzione ad un problema di ricerca

Una sequenza di azioni forma un **cammino**

Una **soluzione** è un cammino che porta dallo stato iniziale a uno stato obiettivo.

Assumiamo che i costi delle azioni siano additivi, cioè che il costo totale di un cammino sia la somma dei costi delle singole azioni. Assumiamo inoltre che tutti i costi delle azioni siano **positivi** (il costo lungo un cammino non può diminuire).

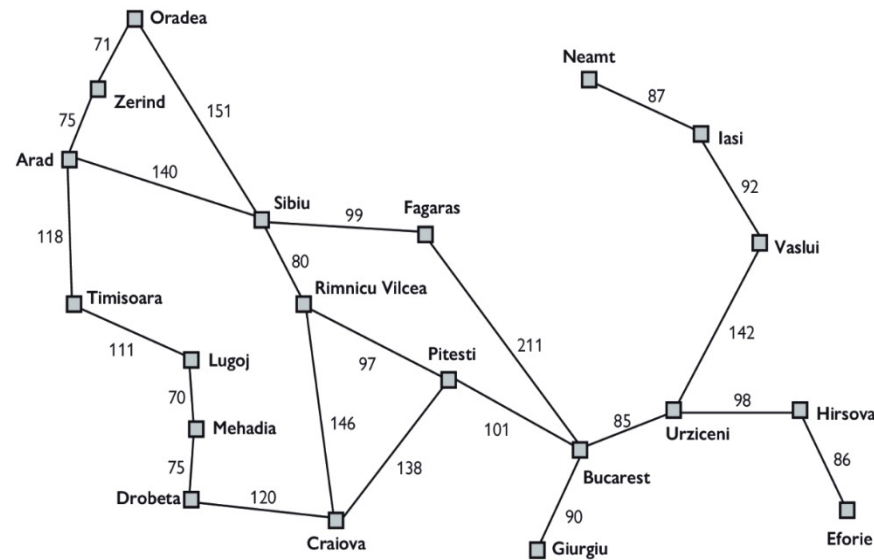
Una **soluzione ottima** è, tra tutte le possibili soluzioni, quella che ha il **costo minimo**.



Spazio degli stati come un grafo

Lo spazio degli stati può essere rappresentato come un **grafo** in cui i vertici (o nodi) rappresentano gli stati e i collegamenti orientati (archi) tra di essi rappresentano le azioni.

La mappa della Romania è un grafo di questo tipo, dove ogni strada indica due possibili azioni di spostamento, una per ogni direzione.



Esempio: il mondo dell'aspirapolvere

Stati: uno stato del mondo indica quali oggetti sono in quali celle in questo caso l'**agente** (aspirapolvere) e lo **sporco**. Nella versione semplice a due riquadri (celle) ci sono 8 stati. Un mondo dell'aspirapolvere con n celle ha $n \times 2^n$ stati.

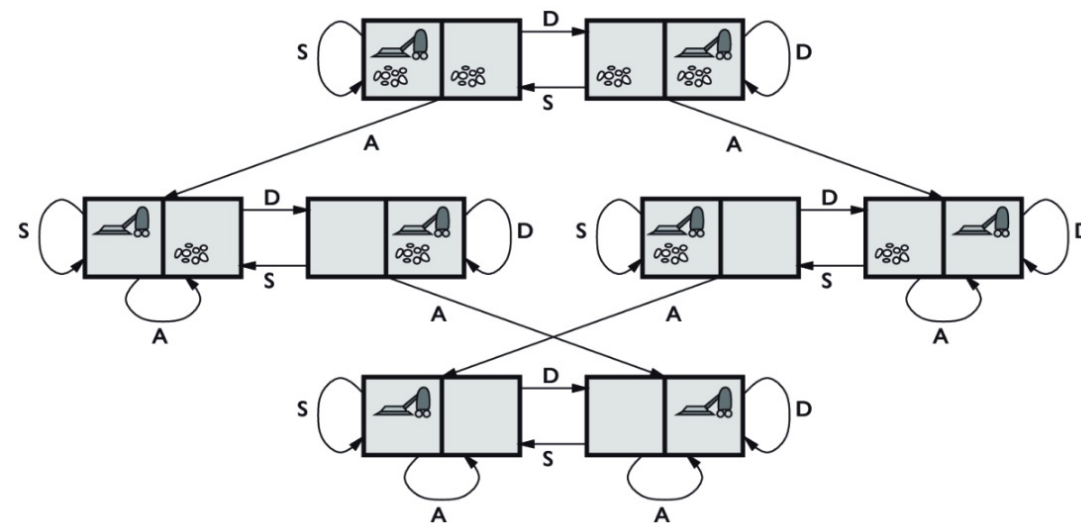
Stato iniziale: uno qualunque.

Azioni: nel mondo a due celle abbiamo definito tre azioni: Sinistra, Destra e Aspira.

Modello di transizione: **Aspira**: rimuove lo sporco dalla cella dove si trova l'agente; **Sinistra\Destra** muove l'agente nella cella a sinistra\destra a meno che non ci sia un muro, nel qual caso l'azione non ha effetto.

Stati obiettivo: gli stati in cui ogni cella è pulita.

Costo di azione: ogni azione costa 1.



Esempio: rompicapo dell'8

Stati: una descrizione di stato specifica la posizione di ognuno degli otto tasselli.

Stato iniziale: ogni stato può essere designato come stato iniziale.

Azioni: lo spazio vuoto si muove a **Sinistra**, a **Destra**, **Su** o **Giù**. Se lo spazio vuoto si trova su un bordo o su un angolo, non tutte le azioni sono applicabili.

Modello di transizione: fa corrispondere a uno stato e un'azione lo stato risultante; per esempio, se applichiamo Sinistra allo stato iniziale della figura, lo stato risultante prevede il tassello 5 e lo spazio vuoto scambiati di posto.

Stato obiettivo: uno stato con i numeri ordinati non separati da uno spazio

Costo di azione: ogni azione costa **1**.

7	2	4
5		6
8	3	1

Stato iniziale

	1	2
3	4	5
6	7	8

Stato obiettivo

7	2	4
	5	6
8	3	1

Cercare Soluzioni

L'idea è quella di mantenere ed estendere un insieme di sequenze (**soluzioni parziali**).

Un agente con diverse opzioni di esito sconosciuto può decidere cosa fare esaminando prima le differenti sequenze possibili di azioni che conducono a stati di esito conosciuto scegliendo, poi, quella migliore.

- Il processo di cercare tale sequenza è chiamato **RICERCA**.

È utile pensare al processo di ricerca come la costruzione di un **albero di ricerca** i cui nodi sono stati e i cui rami sono **azioni**.

Un **algoritmo di ricerca** riceve in input un problema di ricerca e restituisce una **soluzione** o un'indicazione di **fallimento**.

Una volta che viene trovata la soluzione, le azioni suggerite possono essere realizzate.

- Questa fase è chiamata **ESECUZIONE**.



Algoritmi di ricerca

Un **algoritmo di ricerca** riceve in input un problema di ricerca e restituisce una **soluzione** o un'indicazione di **fallimento**.

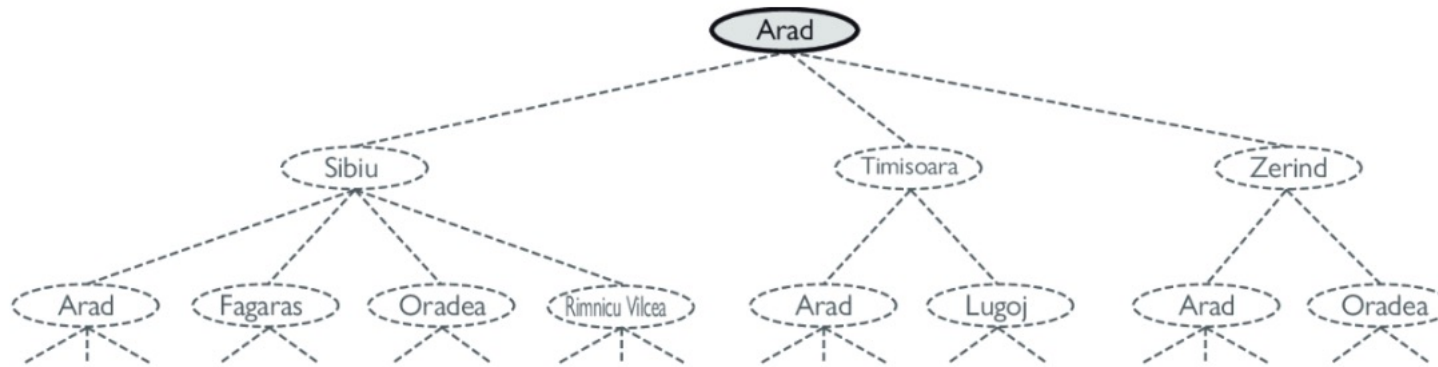
Vedremo algoritmi che effettuano la ricerca di una soluzione sovrapponendo un **albero di ricerca** al grafo dello spazio degli stati, formando vari cammini a partire dallo stato iniziale e cercando di trovarne uno che raggiunga uno stato obiettivo.

Ciascun nodo nell'**albero di ricerca** corrisponde a uno stato nello spazio degli stati e i rami dell'albero di ricerca corrispondono ad **azioni**. La radice dell'albero corrisponde allo **stato iniziale** del problema.

La ricerca ha un **costo**, l'agente deve decidere quali risorse dedicare alla ricerca e quali all'esecuzione.



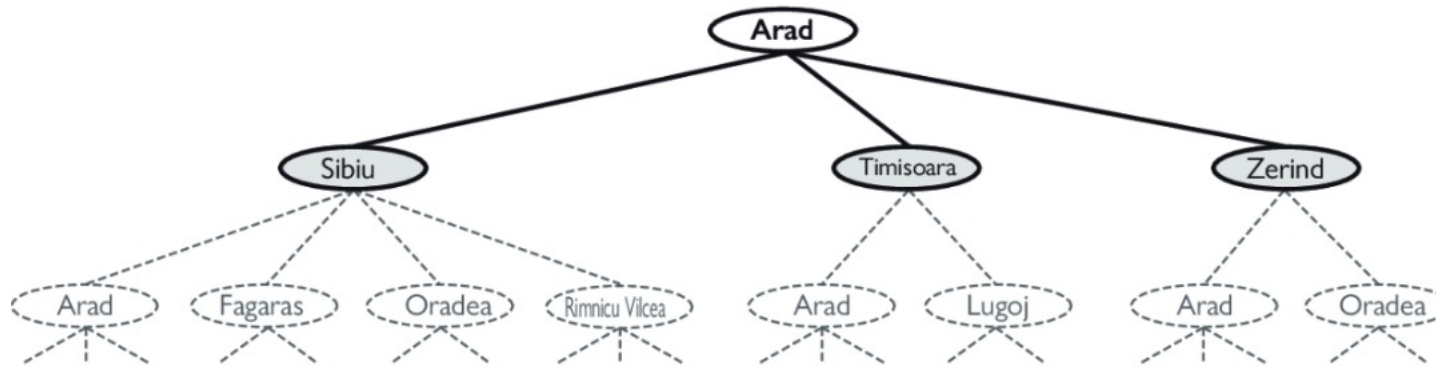
Albero di ricerca: esempio del viaggio in Romania



La Figura mostra i primi passi nella ricerca di un cammino che porti da **Arad** a **Bucarest**.

Il nodo radice dell'albero di ricerca corrisponde allo stato iniziale, **Arad**.

Albero di ricerca: esempio del viaggio in Romania

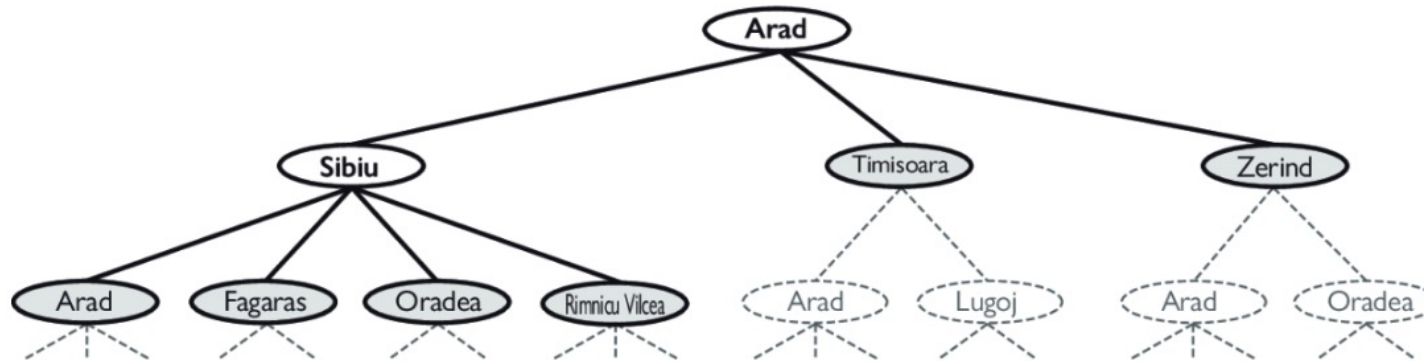


Si può decidere di **espandere** un nodo considerando le Azioni disponibili per quello stato usando la funzione **Risultato(s, a)** per vedere dove portano tali azioni e **generare** un nuovo nodo (chiamato **nodo figlio** o **nodo successore**).

L'espansione del nodo **Arad** genera tre nodi: **Sibiu**, **Timisoara** e **Zerind**.

Ogni nodo figlio ha **Arad** come **nodo padre**

Albero di ricerca: esempio del viaggio in Romania



Nella Ricerca dobbiamo poi scegliere quale dei nodi figli considerare mettendo da parte le altre opzioni, pronti a riprenderle in seguito.

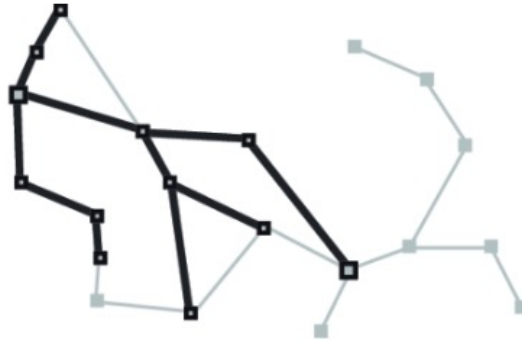
Supponiamo di scegliere di espandere prima **Sibiu**. Questo ci porta ad avere un insieme di 6 nodi non espansi (evidenziati in grigio). Chiamiamo l'insieme di questi nodi **frontiera** dell'albero di ricerca.

La **frontiera** è l'insieme dei nodi (corrispondenti stati) che sono stati **raggiunti** ma non ancora **espansi**

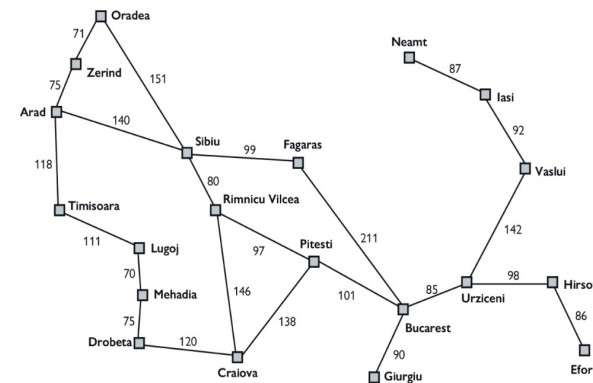
Albero di ricerca vs Spazio degli stati

È importante comprendere la distinzione tra **spazio degli stati** e **albero di ricerca**:

- Lo **spazio degli stati** descrive l'insieme (eventualmente **infinito**) degli stati nel mondo e le azioni che consentono le transizioni da uno stato a un altro.
- L'albero di ricerca descrive cammini tra questi stati per raggiungere l'obiettivo.



Albero di ricerca



Spazio degli stati

Esempio di albero di ricerca generato sul grafo del problema Viaggio in Romania

Strategie di ricerca

Come decidiamo quale nodo della frontiera espandere?

Il **problema** determina il grafo di rappresentazione degli stati e l'obiettivo, ma non definisce la modalità di selezione di un nodo o come aggiungere un nodo alla frontiera dell'esplorazione.

Una *strategia di ricerca* specifica quali elementi devono essere selezionati nella frontiera;

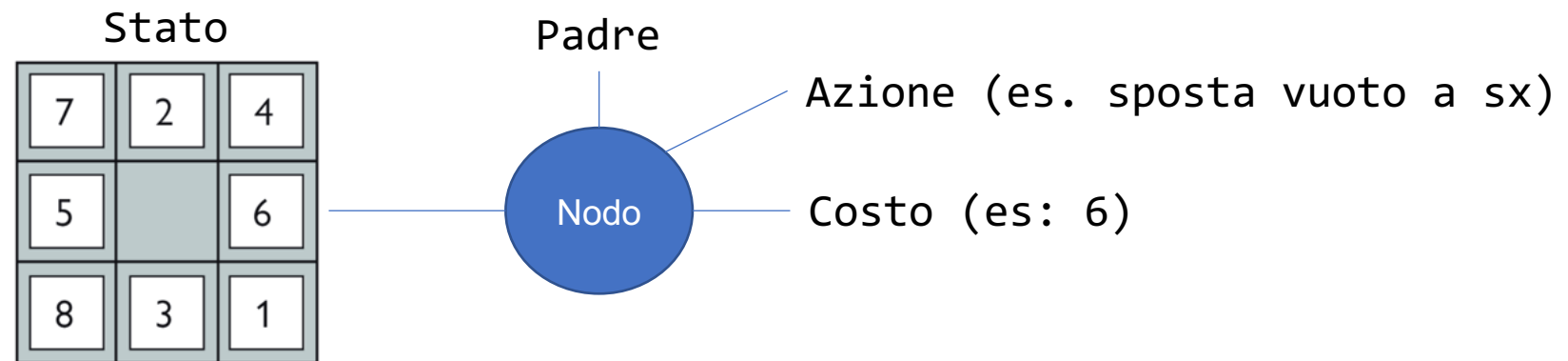
Strategie diverse di ricerca si differenziano per le modalità di selezione e di aggiunta di un nodo alla frontiera.



Strutture dati per la ricerca: il nodo

Gli algoritmi di ricerca richiedono una **struttura dati** per tenere traccia dell'albero di ricerca. Un nodo **n** nell'albero di ricerca è rappresentato da una struttura dati con quattro componenti:

- **n.Stato**: lo stato a cui corrisponde il nodo
- **n.Padre**: il nodo dell'albero di ricerca che ha generato il nodo corrente
- **n.Azione**: l'azione applicata allo stato del padre per generare il nodo corrente
- **n.Costo-Cammino**: il costo totale del cammino che va dallo stato iniziale al nodo corrente.



Strutture dati per la ricerca: la frontiera

La frontiera è rappresentata come una **coda** su cui possiamo svolgere le seguenti operazioni:

- **Vuota?(frontiera)**: restituisce **true** (vero) solo se non ci sono più nodi nella frontiera.
- **Pop(frontiera)**: rimuove il nodo in cima alla frontiera e lo restituisce.
- **Top(frontiera)**: restituisce (senza rimuoverlo) il nodo in cima alla frontiera.
- **Aggiungi(nodo, frontiera)**: inserisce un nodo al posto appropriato nella coda.

Tre tipi di code:

- **coda con priorità**: viene estratto prima il nodo di costo minimo in base alla funzione di valutazione **f**.
- **coda FIFO** (*First-In, First-Out*) il primo nodo aggiunto è il primo ad essere estratto.
- **coda LIFO** (*Last-In, First-Out*) detta anche **stack**: il primo nodo estratto è l'ultimo ad essere stato aggiunto alla coda.



La strategia best-first

Un approccio generale è quello della ricerca **best-first** (“prima il migliore”), in cui scegliamo un nodo **n** a cui corrisponde il valore minimo di una funzione di valutazione **f(n)**.

Seguendo questa idea, una ricerca consiste nel:

1. scegliere ad ogni iterazione un nodo sulla **frontiera** in cui **f(n)** ha valore **minimo** e:
2. lo restituiamo se il suo stato è uno stato obiettivo,
3. altrimenti applichiamo **Espandi** per generare nodi figli
4. ogni suo nodo figlio viene aggiunto alla **frontiera** se: (i) non è stato raggiunto in precedenza, o (ii) è stato raggiunto con un cammino di costo inferiore ai precedenti
5. l'algoritmo restituisce un'indicazione di **fallimento** oppure un nodo che rappresenta un cammino che porta a un obiettivo.

Utilizzando funzioni **f(n)** diverse otteniamo algoritmi diversi



Algoritmo generale di ricerca (Best-first)

```
function Ricerca-Best-First(problema, f) returns un nodo soluzione o fallimento  
  nodo  $\leftarrow$  Nodo(Stato=problema.StatoIniziale)  
  frontiera  $\leftarrow$  una coda con priorità ordinata in base a f, con nodo come elemento iniziale  
  raggiunti  $\leftarrow$  una tabella di lookup, con un elemento con chiave problema.StatoIniziale e valore nodo  
  while not Vuota?(frontiera) do  
    nodo  $\leftarrow$  Pop(frontiera)  
    if nodo.Stato == problema.Obiettivo then return nodo  
    for each figlio in Espandi(problema, nodo) do  
      s  $\leftarrow$  figlio.Stato  
      if s not in raggiunti or figlio.Costo-Cammino < raggiunti[s].Costo-Cammino then  
        raggiunti[s]  $\leftarrow$  figlio  
        aggiungi figlio a frontiera  
  return fallimento
```



La funzione Espandi

La funzione **Espandi** prende in ingresso il *problema* ed un *nodo* e restituisce un insieme di **nodi successore** del *nodo* in input

```
function Espandi(problema, nodo) return un insieme di nodi
  s ← nodo.Stato
  successori ← emptyset
  for each azione in problema.Azioni(s) do
    s' ← problema.Risultato(s, azione)
    costo ← nodo.Costo-Cammino + problema.Costo-Azione(s, azione, s')
    successori.add(new Nodo(Stato=s', Padre=nodo, Azione=azione, Costo-Cammino=costo)
  return successori
```



Prestazioni di un algoritmo di ricerca

Possiamo valutare le prestazioni di un algoritmo secondo quattro parametri.

- **Completezza**: l'algoritmo garantisce di trovare una soluzione quando esiste, e di riportare correttamente il fallimento quando la soluzione non esiste?
- **Ottimalità rispetto al costo**: trova la soluzione con il costo di cammino minimo fra tutte?
- **Complessità temporale**: quanto tempo impiega a trovare una soluzione? Il tempo può essere misurato in secondi o, in modo più astratto, in base al numero di stati e azioni considerato.
- **Complessità spaziale**: di quanta memoria necessita per effettuare la ricerca?



Tipi di ricerca

A seconda del tipo di conoscenza che possiede o riceve l'agente possiamo dividere i **problemi di ricerca** in:

- **Ricerca non informata (o cieca):** Non si ha alcuna informazione sul numero di passi o sul costo di **cammino dallo stato attuale all'obiettivo**
- **Ricerca informata (euristica):** Si hanno informazioni di preferenze tra gli stati

Come vedremo, la ricerca informata è più efficace di quella non informata

